GRAPHS

18.1 GRAPH

A graph is a non linear data structure, which contains a set of vertices or nodes and edges which connect themselves. In case of tree structures there is a hierarchical relationship between parent and children i.e. one parent and many children. On the other hand in case of a graph, the relationship is from many parents to many children.

This graph theory is originated in koningsberg bridge problem. The prussian city of Kongsberg was built along the Preger River and occupied both banks and two islands. Here the problem was to seek a route that would enable one to cross all seven bridges in the city exactly once and return to their starting point. Leonhard Euler, a mathematician developed the concepts for solving the problem, which formed the basis of graph theory.

18.2 Graph Terminologies:

There are number of important terminologies associated with the theory of graphs. They are:

Graph

A graph G = (V, E) consists two sets, where V is the set of vertices and E is the set of edges.

<u>Diagram</u>

In the above graph, G= { V, E } where

V= { V_1,V_2,V_3,V_4 }

$E=\{ (V_1, V_2), (V_2,V_3), (V_3,V_4),(V_4,V_1), (V_1,V_3), (V_2, V_4) \}$

Directed Graph (Diagraph)

<u>Diagram</u>

It is a graph such that G= {V, E} where V is set of vertices and E is the set of ordered pair of vertices.

$V= \{ V_1,V_2,V_3,V_4 \}$
$E=\{ (V_1, V_2), (V_2,V_3), (V_3,V_4),(V_4,V_1), (V_1,V_3), (V_2, V_4) \}$

If an ordered pair (V_i, V_J) is in set E, then there is an edge directed from V_i to V_J (indicated by arrow sign).

In case of undirected graph a pair (V_i, V_J) is in set E is unordered. i.e. (V_i , V_J)and (V_J, V_i)are the same edges, but in directed graph they refer to two different edges.

Weighted Graph

A graph is termed as weighted if all the edges are labeled with some weights. A weighted graph can be either directed/undirected.

<u>Diagram</u>

Adjacenct Vertices

<u>Diagram</u>

In case of directed graph, the vertex V_i is adjacent to another vertex V_J, if there is an edge from V_i to V_j. In the above graph V_2 is adjacent to V_4 and V_3. V_1 is not adjacent to V_4 but adjacent to V_2.

Self Loop
<u>Diagram</u>

If there is an edge exist, where starting and end vertex are same i.e. (v_i, v_i) is an edge , then it is called a self loop.

Parallel edges
Diagram

In a graph, It there are more than one edge between the same pair of vertices, then they are known as the parallel edges. In the above graph parallel edges exist for the pair of vertices V_1,V_4.

Here, these are two parallel edges between 20 and 30.

Simple Graph

A graph which does not have parallel edges and self loops, is called simple graph.

Multi-graph

A graph which contains either a self loop or parallel edges or both, is referred as multi-graph.

Diagram

Complete Graph

A graph G is said to be complete if there is a path from each vertex to all other vertices of the graph.

<u>Diagram 1</u> **<u>Diagram 2</u>**

Cycle

If there is a path containing edges which start from vertex V_i and ends at vertex V_i , then the path is called cycle.

Cyclic Graph

A graph with one or more cycles is known as cyclic graph.

<u>Diagram</u>

Acyclic Graph

A Graph which does not have cycles or cyclic paths, then it is called Acyclic graph. Tree is an acyclic graph.

<u>Diagram</u>

Isolated Vertex

A vertex is isolated if there is no edge connected from any other vertex to that vertex.

<u>Diagram</u>

Connected Graph

A graph is called connected, if there exist a path from any vertex to any other vertex (or) if for every pair of distinct vertices V_i, V_j. In graph G, there is a path, then it is connected graph. A directed graph with this property is called strongly connected; otherwise it is called weakly connected.

Diagram 1 **Diagram 2**

Degree of a Node/Vertex

For undirected graph, the number of edges connected with vertex V_i, is considered degree of vertex V_i. For example, in the following undirected graph, degree of V_1 is 3.

Diagram 1

For directed graph, there are two types of degrees. They are:

1. indegree
2. outdegree

1. Indegree: Indegree of a vertex V_i is number of edges incident into V_i (i.e. number of edges ending at V_i).
2. Outdegree : Outdegree of a vertex V_i is number of edges eminating from V_i (i.e. number of edges beginning at V_i)

Diagram

For example, in the above directed graph,

Indegree of $V_1 = 1$ (i.e. V_4)

Outdegree of $V_1 = 2$ (i.e.V_2, V_3)

Similarly, Indegree of $V_2 = 1$ and Outdegree of $V_2 = 2$

Indegree of $V_3 = 2$ and Outdegree of $V_3 = 1$

Indegree of $V_4 = 2$ and Outdegree of $V_4 = 1$

Pendant Vertex

A vertex V_i is called pendant if indegree of V_i is 1 and outdegree of V_i is 0.

REPRESENTATION OF GRAPH

19.1 Representation of Graph:

While representing a graph, we need to find the mathematical representation of graph i.e. in terms of sets. The graph then can be represented using

1. Sequential/Matrix representation
2. Linked List Representation

19.1.1 Matrix Representation of Graph

Matrix representation of graph can be of two types.

1. Adjacency Matrix
2. Incidence Matrix

19.1.1.1 Adjacency Matrix:

For a Graph 'G' with N number of vertices, a matrix of order N x N order can be taken.

Diagram

Here, if 'i' represents row position and 'j' represents column position, then if there is an edge from V_i to V_j, then A[i][j] =1 else A[i][j] = 0.

Diagram

V_1	V_2	V_3	V_4	V_5
0	**0**	**0**	**0**	**1**
1	**0**	**0**	**0**	**1**
0	**1**	**0**	**1**	**1**
0	**0**	**0**	**0**	**0**
0	**0**	**0**	**1**	**0**

It is a simple way to represent a graph in the form of matrix, and the matrix is called Adjacency Matrix.

Disadvantages of Adjacency Matrix

1. It requires n^2 space to represent a graph and also takes n^2 time to solve most of the graph related problems.
2. It may be difficult to insert and delete nodes because, the size of matrix need to be changed and at the same time the sequence of the nodes should be recorded and nodes may be needed to reordered.

19.1.1.2 Incidence Matrix

An incidence matrix consists of a row for every vertex and a column for every edge.

Here if there is an edge E_k exist from V_i to V_j, then V_iE_k=1 and V_j E_k=-1 and others at Ek =0. So, here for the edge E_k, V_i and V_j are adjacent nodes.

The incidence matrix for the above directed graph G is as follows.

19.1.2 Linked List Representation of Graph

While representing a graph in linked list format the structure of the node will be as follows.

Next	Vertex	List

Here every vertex consists of 3 parts. i.e.

1. Next : It is a pointer to the next vertex in the graph.
2. Vertex : It consists of the vertex name
3. List : It points to the 1st node of adjacency list of a vertex.

In case of directed graph it points to the successor nodes (or) adjacency nodes connected with outdegree edges.

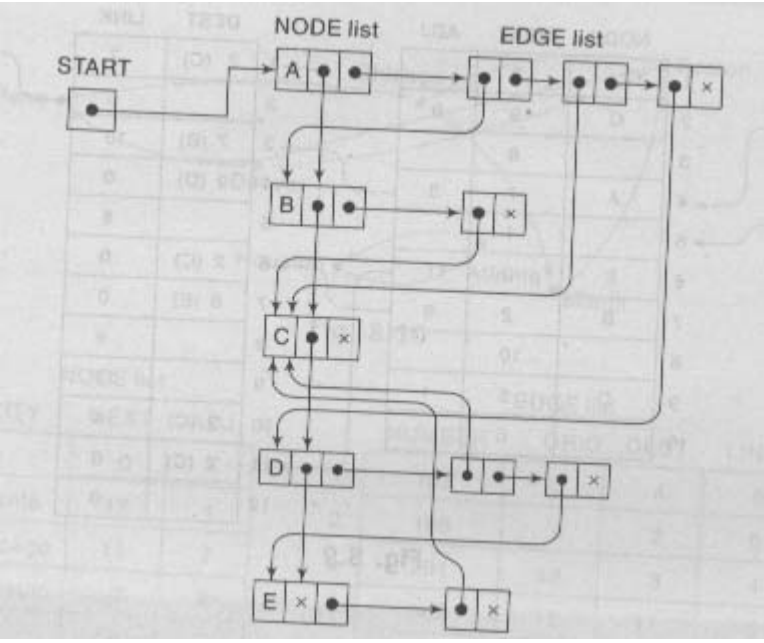
<u>Diagram</u>

The adjacency list for the above graph is :

Node	Adjacency List
A	B, C, D
B	C
C	
D	C, E
E	C

The list representation contains (1) Nodes List (2) Edges List

Linked Representation of above Graph is as follows.



Path Matrix or Reachability Matrix

A path matrix contains information about each vertex to other vertex, there exist a path or not consider a graph G:

Table

Adjacency matrix for the graph is

Table

From Adjacency Matrix, we can know, there exists a direct single edge from one node to another or not.

Consider A^2, A^3, A^4 (according to the size of matrix /no. of vertices)

Table

Here A^2 represent how many paths exist for a particular node to another node, using two edges.
e.g. in A^2 Matrix. V_2 to V_4 is having two paths using two edges.

Table

A^3 shows number of paths exist using 3 edges.

Table

A^4 shows no. of paths exist using 4 edges.
$B_N=A+A^2+A^3+\ldots..+A^N$ Where N is the total number of nodes
So, $B_4 = A + A^2 + A^3 + A^4$

Table

Now by representing all non zero entries by 1, we obtain path matrix :
Table

So here P is the path matrix that shows whether there exist a path from one node to other node or not.

SHORTEST PATH PROBLEMS

20.1 Shortest Path Problems

The problem in a graph is about finding a path between two vertices in such a way that this path will satisfy some conditions of optimization. In a non weighted graph, to find a shortest path, we count the no of edges. In a weighted graph, the shortest path is determined by sum of the weights on edges in a path.

To find these details there are standard algorithms used i.e.

- Warshalls algorithm,
- Floyd's algorithm
- Dijkstra's algorithm

20.1.1 Warshall's Algorithm (For path matrix)

Warshall's algorithm is basically used to find the path matrix for a given graph. By using Warshall's algorithm one can specify whether there exists a path between two vertices or not.

Assume a directed graph G with N nodes is maintained in memory by its adjacency matrix A. The algorithm finds path matrix P of graph G.

Algorithm:
[Assume A is an adjacency matrix of order N x N order. This algorithm find the path matrix P]

```
Step 1: Let I, J, K
Step 2:         Repeat For I: =1 to N incrementing by 1
        Step 2.1: Repeat For J: =1 to N incrementing by 1
                Step 2.1.1: If A [I] [J] =0, Then
                                Set P [I] [J]:=0
                Step 2.1.2: Else
                                Set P [I] [J]:=1
                            [End of If]
                  [End of For]
        [End of For]
Step 3: Repeat For K: =1 to N incrementing by 1
        Step 3.1: Repeat For I: =1 to N incrementing by 1
```

```
            Step 3.1.1: Repeat For J: =1 to N incrementing by 1
                        P [I] [J]:=P [I] [J] U (P [I] [K] ∩ [P] [K] [J])
                    [End of For]
              [End of For]
        [End of For]
Step 4: Exit
```

20.1.2 Floyd's Algorithm (All pair shortest path problem)

By using Warshall's algorithm one can find path matrix which shows presence or absence of any path between a pair of vertices. If a weighted graph exists, and we are interested to find the length of shortest path between any pair of vertices, then we follow Floyd's algorithm for this purpose.

Algorithm:
[Assume W is a matrix of NxN order that represents a weighted graph. This finds a matrix Q of NxN order which represents the shortest path from any vertex to any vertex]

```
Step 1: Let I, J, K
Step 2:     Repeat for I:=1 to N incremented by 1
        Step 2.1: Repeat, for J: =1 to N incremented by 1
            Step 2.1.1: If W [I] [J] =0, then
                        Set Q0 [I] [J]:= ∞
            Step 2.1.2: Else
                        Set Q [I] [J]:=W [I] [J]
                    [End of If]
              [End of For]
        [End of For]
Step 3:     Repeat For K: =1 to N incremented by 1
        Step 3.1: Repeat For I: =1 to N incremented by 1
            Step 3.1.1: Repeat For J: =1 to N incremented by 1
                        Set Q[I][J]:=MIN( Q[I][J], Q[I][K] + Q[K][J] )
                    [End of For]
              [End of For]
        [End of For]
Step 4:     Exit
```

<u>Floyd's Algorithm – Dry RUN</u>

Table

Table

Using the algorithm we obtain Q_0, Q_1, Q_2, Q_3 and Q (Where Q_0 represents single edge weights)

Table

(At Q_1 is the result of minimum weight between (each weight of Q_0 and each weight of two edges using R)).

Table

e.g. at Q_0, S-S using single edge, so now S-R-S using two edge 12. So minimum distance is 12, updated at Q_1.
Q_0 is the result of minimum weight between (each weight of Q1 and each weight of edges using S)
Table

e,g At Q_1 T-U is now T-S-U is 5, so minimum distance 5 updated At Q_2.

Table

Q_2 is the result of minimum weight between (each weight of Q_2 and each weight of edges using T).
e.g. at Q_2 U-S is 9, now U-T-S is 4, so minimum distance 4 is updated at Q_3.

Table

Q_4 is the result of minimum weight between (each weight of Q_3 and each weight of edges using U)
e.g. At Q_3 T-R is 10, now T-S-U-R is 9, so minimum 9 is updated at Q_4.

3.1.3. Dijkstra's Algorithm (Single source shortest path problem):

Dijkstra's algorithm is an algorithm for solving the single source shortest path problem in a weighted graph. Here we consider a distinct vertex as source vertex and the algorithm will find the shortest possible path from this source vertex to all other vertices of the graph. So, this algorithm is also called single source shortest path algorithm.

Diagram:

- V is set of vertices. In the above V={1,2,3,4,5,6}
- S Set of source vertices, initially S={1}
- C is Cost/weight of an edge
- N identifies total number of nodes
- D [N] is an array which contains the distance/weight of each vertex from source vertex.
- W is the vertex with minimum weight in the list of { V – S }
- U identifies each vertex of { V – S }

Algorithm

Step 1: V= {1, 2, 3, 4, 5, 6}
Step 2: S = {1}
Step 3: Repeat For *i* = 2 to N incrementing by 1
D[i]:= C (1, *i*)
[End of For]
Step 4: Repeat For *I*: = 1 to N-1 incrementing by 1

Step 4.1: Choose a vertex W in {V – S} such that D [W] is minimum.
Step 4.2: Add W to S
Step 4.3: Repeat For each vertex U in {V – S}
D [U]:=min (D [U], D [W] + C (W, U))
[End of For]
Step 5: Exit.

Process Steps of Dijkstra's Algorithm

V = {1, 2, 3, 4, 5, 6}
S = {1}
D [N] =

∞	10	90	∞	70	60
1	2	3	4	5	6

i. →

Here consider vertex 1 at cost ∞
Now from vertex 1 to each vertex cost is C(1,*i*)
C (1, 2) i.e. 1 to 2 = 10
C (1, 3) i.e. 1 to 3 = 90
C (1, 4) i.e. 1 to 4 = ∞
C (1, 5) i.e. 1 to 5 = 70
C (1, 6) i.e. 1 to 6 = 60

When *i*=1

Step 4

∞	20	90	∞	70	60
1	2	3	4	5	6

i →

Step 4.1 V – S = {2, 3, 4, 5, 6}
Now identify minimum cost D[W] from 2 to 6 positions.
i.e. W = 2
Step 4.2. Add W i.e. 2 in to S . So, S = {1.2}
Step 4.3

∞	10	90	∞	70	60
1	2	3	4	5	6

U →

When U=3

D [3] = min (D[3], D[2] + C(2,3))
= min (90, 10+20)
= min (90, 30)
So D [3] = 30

∞	10	30	∞	70	60
1	2	3	4	5	6

Step 4

When *i* =2

V = {3, 4, 5, 6}

1) Identify minimum cost D [W] from 3 to 6 i.e. minimum cost is 30 i.e. W=3

2) Add W in to S = {1, 2, 3}
So V-S = {4, 5, 6}

When U = 4

Now D[4] = min (D[4], D[3]+C(3,4))
= min(∞, 30 + 30)
= 60
So D[4]=60

∞	10	30	60	70	60
1	2	3	4	5	6

Step 4

When *i* =3

V = {4, 5, 6}

1) Identify minimum cost D [W] from 4 to 6 i.e. minimum cost is 60 i.e. W=4

2) Add W to S. So, S = {1, 2, 3, 4} and V – S = {5, 6}
Now D [U] = min (D [U], D [W] + C (W,U))
i.e. D[5] = min(D[5], D[4] +C(4,5))
= min (70, 60+∞)
= 70
So D[5] = 70

∞	10	30	60	70	60
1	2	3	4	5	6

Step 4

When *i* = 4

V= {5, 6}

1) Identify minimum cost D[W] form 5 to 6 so minimum cost is 60 i.e. W=5
2) Add W to S. So, S ={1,2,3,4,5} and V-S ={6}

   Now D[6] = min (D[6], D[5]+C[5,6])

   = min (60, 70 +∞)

   = 60

So

∞	10	30	60	70	60
1	2	3	4	5	6

GRAPH TRAVERSALS

21.1 Traversing a Graph

A graph can be traversed in two standard ways. They are:

1. Depth First Search (DFS)
2. Breadth First Search (BFS)

In depth-first search, we will take the help of a stack and in breadth-first search we will take the help of a queue to hold the nodes for future processing.

During the execution of above traversal algorithms we consider three states for each vertex by taking a variable Flag, where

Flag =1 represents Ready State
Flag =2 represents Waiting State
Flag =3 represents Processed State

21.1.1 Depth First Search(DFS)

Algorithm:

[Assume a graph G with nodes. This algorithm will traverse the by in depth first order by using a stack]

Step 1: [Initialize all nodes to ready state]
Set flag = 1 for all the nodes.
Step 2: [Consider a node as 1st node and push into stack]
PUSH (1)
Step 3: [change the status of node 1 to waiting state]
Set flag = 2 for node 1
Step 4: Repeat While STACK is Not Empty
Step 4.1: Set x = POP ( )
Step 4.2: Process x
Step 4.3: [change status of x into Processed State]
Set flag = 3 for node x.
Step 4.4: Repeat for all adjacent nodes of x (say y)
Step 4.4.1: If flag =1 for node y, then
PUSH(y)

Set flag = 2 for node y
[End of If – Step 4.4.1]
[End of For – Step 4.4]
[End of while – Step 4]
Step 5: Exit

<u>Dry Run of DFS:</u> (A simple graph without cycles)

Table

1. (1,2,3,4,5) Flag – 1 (ready state)

2. Ready State (2,3,4,5)
   Waiting State (1)
   STACK (1)

3. Ready State (5)
   Waiting State (2, 3, 4)
   Processed State (1)
   STACK (2,3,4)

4. Ready State ( )
   Waiting State (2,3,5)
   Processed State(1,4)
   STACK(2,3,5)

5. Ready State ( )
   Waiting State (2,3)
   Processed State (1,4,5)

STACK(2,3)

6. Ready State ( )
   Waiting State (2,5)
   Processed State (1,4,5,3)
   STACK(2)

7. Ready State ( )
   Waiting State ( )
   Processed State (1,4,5,3,2)
   STACK ( )

   Traversing is completed as the stack become empty.

   Sequence of vertex in above graph for DFS are 1, 4 ,5 ,3 ,2.

21.1.2 Breadth First Search (BFS)

Algorithm:
[Assume a graph G with nodes. This algorithm will traverse the by in breadth first order by using a queue]

```
Step 1:     [Initialize all nodes to ready state]
        Set flag: = 1 for all nodes.
Step 2:     [Consider a node as 1st node and insert it into Queue]
        Insert (1)
Step 3:     [Change the status of node 1 to waiting state]
        Set flag: =2 for node 1
Step 4:     Repeat while Queue is Not Empty
        Step 4.1: Set x: = Delete ( )
        Step 4.2: Process (x)
        Step 4.3: [Change the status of x into Processed State]
                    Set flag: = 3 for node x.
        Step 4.4: Repeat for each adjacent node y of x
                Step 4.4.1: If flag = 1 for node y, then
                            Insert (y)
                            Set flag: = 2 for node y
                        [End of If – Step 4.4.1]
                [End of For – step 4.4]
        [End of while – Step 4]
```

Step 5: Exit

1. (1,2,3,4,5) Flag – 1 (ready state)

2. Ready State (2, 3, 4, 5)
   Waiting State (1)
   QUEUE (1)

3. Ready State (5)
   Waiting State (2, 3, 4)
   Processed State (1)
   QUEUE (2, 3, 4)
4. Ready State (5)
   Waiting State (3, 4)
   Processed State(1, 2)
   QUEUE (3, 4)

5. Ready State (5)
   Waiting State (4)
   Processed State (1, 2, 3)
   QUEUE (4)

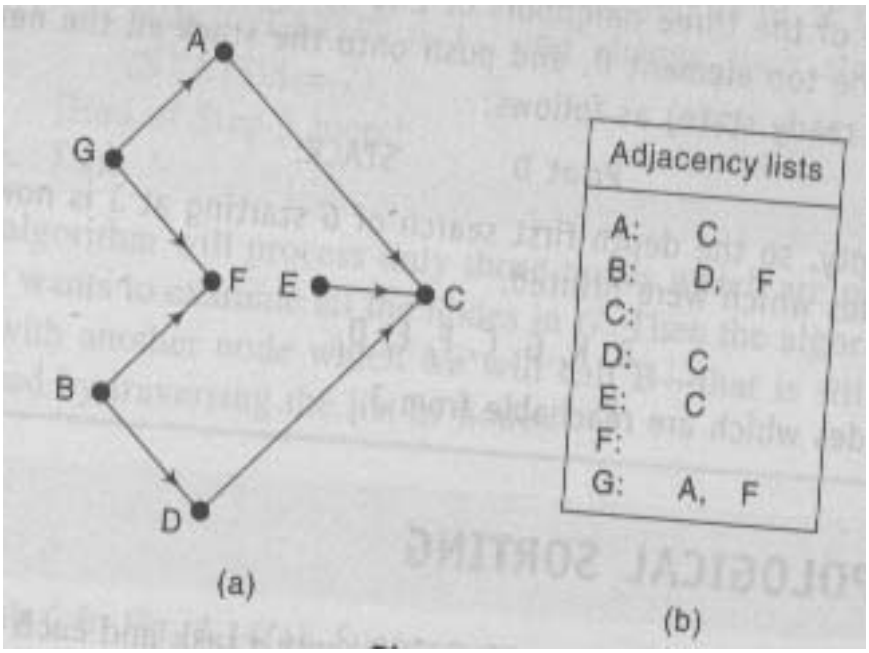
6. Ready State (___)
   Waiting State (5)
   Processed State (1, 2, 3, 4)
   QUEUE (5)

7. Ready State (___)
   Waiting State (___)
   Processed State (1, 2, 3, 4, 5)
   QUEUE(EMPTY)

21.2 Topological Sorting of Graph

In a graph if we assume each node as a specific task. If there exist cycles in a graph, then the tasks in cycles will never get completed. So in this situation graph must be considered as an acyclic graph.

For performing the tasks serially, an acyclic directed graph can be represented in linear order of nodes, which is called topological sorting.



So if a graph is finite directed graph without cycles, then there exists a topological sort.

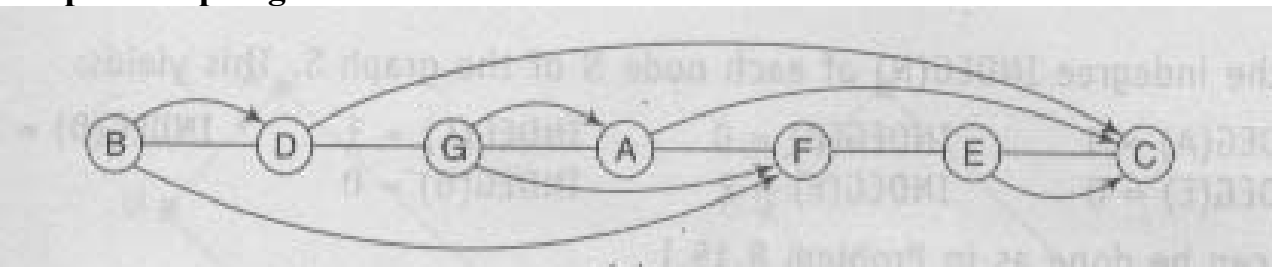
Algorithm:

[Consider a graph G without cycles, this algorithm finds the topological order of the graph G]

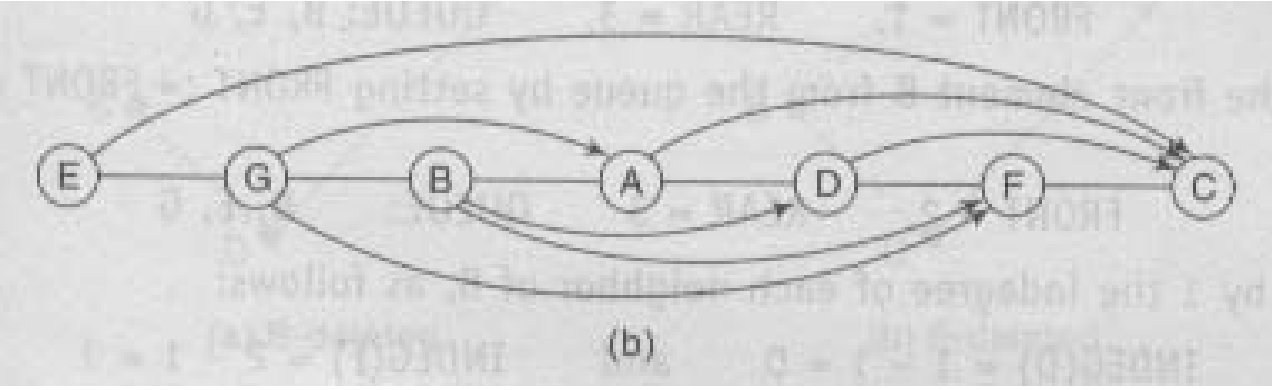
Step 1: Find the indegree (n) of each node 'n' of graph G.
Step 2: Insert all the nodes with 0 indegree into Queue.
Step 3: Repeat until Queue is Empty
    Step 3.1: Remove the front node 'n' of the Queue.
    Step 3.2: Repeat for each neighbour 'm' of node 'n'
        Step 3.2.1: Set indegree (m): =indegree(m) – 1

Step 3.2.2: If indegree(m)=0, then insert it at the Rear end of Queue.
[End of For – Step 3.2]
[End of Loop – Step 3]
Step 4: Exit

Graph in Topological order:



(or)



<u>Dry Run For Topological Sort</u>

Consider the above graph G is for topological sorting. Finally the array T will contain nodes in sorted order. Q is the Queue for this sorting process.

The sorting process as follows:

1. Find the indegree of each node i.e.
   INDEG(A) = 1 INDEG(B) = 0 INDEG(C) = 3 INDEG(D) = 1
   INDEG(E) = 0 INDEG(F) = 2 INDEG(G) = 0

2. Initially add the nodes having zero indegree into the QUEUE

FRONT = 1 REAR = 3 Q = B, E, G

3. Remove the front element B from Q and add to T.
FRONT = 2 REAR = 3 Q = E, G T = B

4. For each neighbor node of B, decrease the indegree by 1
INDEG(D) = 1 - 1 = 0
INDEG(F) = 2 - 1 = 1
Since the indegree of neighbour node D is 0, so it is inserted at the REAR of Q
So, now:
FRONT = 2 REAR = 4 Q = E, G, D T = B

5. Remove the front element E from Q and add to T.
FRONT = 3 REAR = 4 Q = G, D T = B, E

6. For each neighbor node of E, decrease the indegree by 1
INDEG(C) = 3 -1 = 2
Since no zero indegree neighbour nodes exist, so no nodes inserted to Q.

7. Remove the front element G from Q and add to T.
FRONT = 4 REAR = 4 Q = D T = B, E, G

8. For each neighbor node of G, decrease the indegree by 1
INDEG(A) = 1 - 1 = 0
INDEG(F) = 1 - 1 = 0
Since the indegree of neighbour nodes A, F are 0, so they are inserted at the REAR of Q
So, now:
FRONT = 4 REAR = 6 Q = D, A, F T = B, E, G

9. Remove the front element D from Q and add to T.
FRONT = 5 REAR = 6 Q = A, F T = B, E, G, D

10. For each neighbor node of D, decrease the indegree by 1
INDEG(C) = 2 - 1 = 1
Since no zero indegree neighbour nodes exist, so no nodes inserted to Q.

11. Remove the front element A from Q and add to T.
FRONT = 6 REAR = 6 Q = F T = B, E, G, D, A

12. For each neighbor node of A, decrease the indegree by 1
INDEG(C) = 1 - 1 = 0
Since the indegree of neighbour node C is 0, so it is inserted at the REAR of Q
So, now:
FRONT = 6 REAR = 7 Q = F, C T = B, E, G, D, A

13. Remove the front element F from Q and add to T.
FRONT = 7 REAR = 7 Q = C T = B, E, G, D, A, F

14. The node F has no neighbour nodes, so no change takes place.

15. Remove the front element C from Q and add to T.
FRONT = 8 REAR = 7 Q = (EMPTY) T = B, E, G, D, A, F, C

16. The node C has no neighbour nodes, so no change takes place.

Now the Q is Empty and the array T contains elements in topological sorted order.

SORTING AND SEARCHING

22.1 SORTING:

Sorting is the process of arranging the data or information in some logical order. That may be ascending or descending or dictionary order (for Characters).

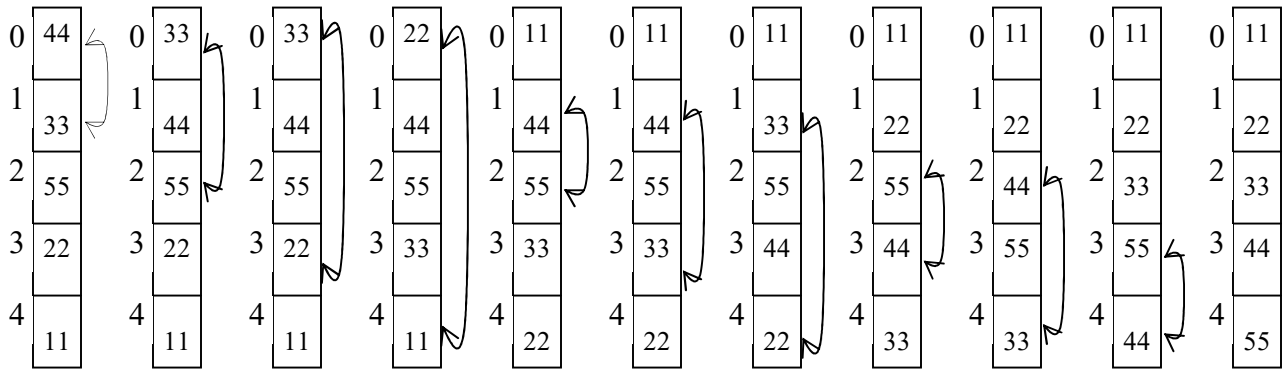
The various sorting techniques that are available are:

1. Selection Sort
2. Bubble Sort
3. Insertion Sort
4. Quick Sort
5. Merge Sort
6. Radix Sort
7. Heap Sort
8. Address Calculation Sort (i.e. Using Hashing)

.

22.1.1 SELECTION SORT:

In selection sort technique, the smallest element in the list is found out by comparing the first element with second, third and so on up to last position. At the end of first pass, the smallest element of the list will be at the 1st position. Now the second smallest element is found out by comparing second element with third, fourth and so on up to last position. At the end second pass, the 2nd smallest element will be at the 2nd position. This process continues for rest of the elements so as to get the list sorted. We require (N-1) number of passes to sort a list of N elements.

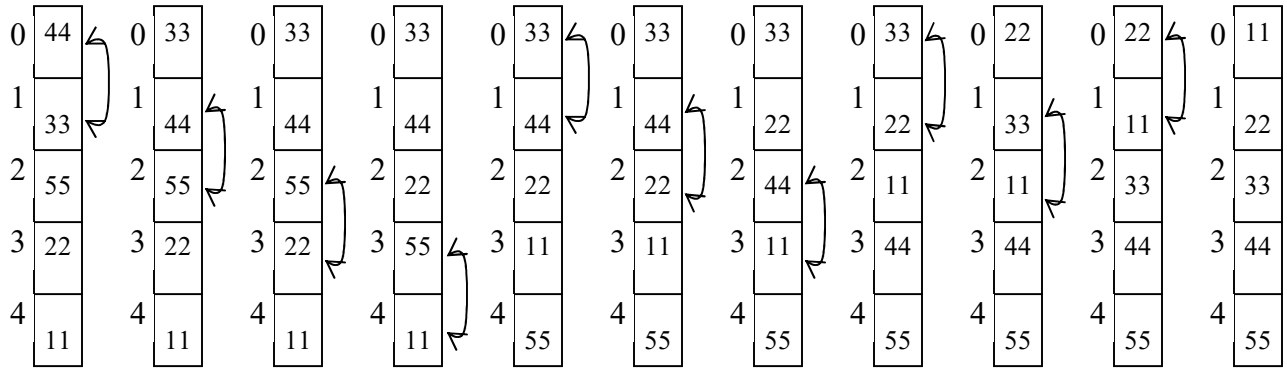


SELECTION_SORT (A [N])

```
Step 1: Let I, J, TEMP
Step 2:        Repeat for I: =0 to N – 1 increasing by 1
        Step 2.1: Repeat for J: =I+1 to N increasing by 1
                Step 2.1.1: if A [I] > A [J], Then
                                TEMP: =A [I]
                                A [I]:=A [J]
                                A [J]:=TEMP
                            [End of If]
                [End of For]
        [End of For]
Step 3: Exit
```

22.1.2 BUBBLE SORT:

In bubble sort, each element is compared with its adjacent element. If the first element is larger than the second one then the position of the elements is interchanged, otherwise it is not changed. Then next element is compared with its adjacent element and the same process is repeated for all the elements in the array. At the end of Pass-1, the largest element in the list will occupy the last position, at the end of Pass-2, the second largest element will occupy the 2nd last position and so on.



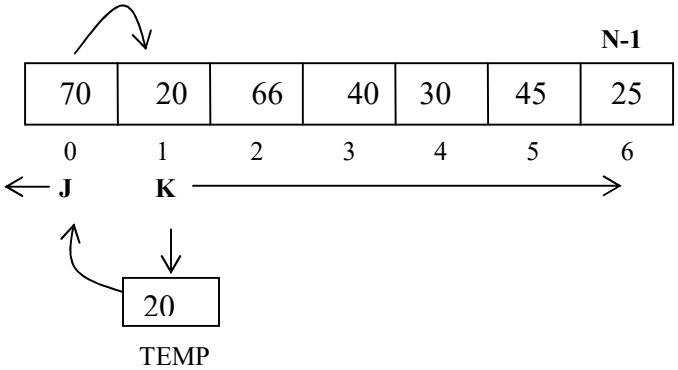
BUBBLE_SORT (A [N])

```
Step 1: Let I, K, TEMP
Step 2:        Repeat for K: =N-2 to 0 decreasing by 1
        Step 2.1: Repeat for I: =0 to K increasing by 1
                Step 2.1.1: If A [I] >A [I +1], then
                                TEMP: =A [I]
                                A [I]:=A [I+1]
                                A [I+1]:=TEMP
                            [End of If]
                [End of For]
        [End of For]
Step 3:        Exit
```

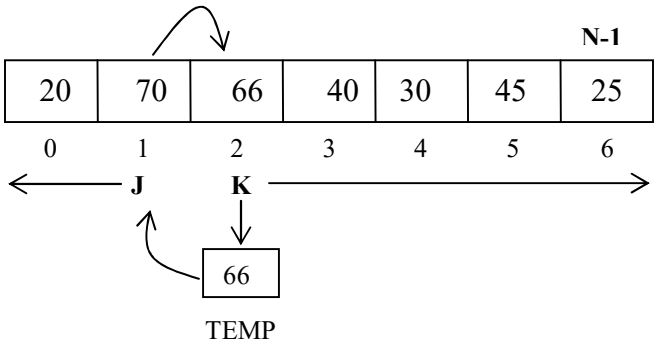
22.1.3 INSERTION SORT

Insertion sort is very much similar in the way of arranging the cards by a card player. The card player picks up the cards and inserts them in the proper position such that the cards on hand are sorted. So, in insertion sort, each element is scanned and it is inserted in the proper position of previously sorted sub-array such that it is in sorted manner. To start with, we will assume that the array contains only one element and it is sorted trivially. Then we scan the second element of the array and it is compared with the first element and based on its value the second element will be kept at the left or right of first element such that two elements are sorted. This procedure is applied to rest of the elements of the array.

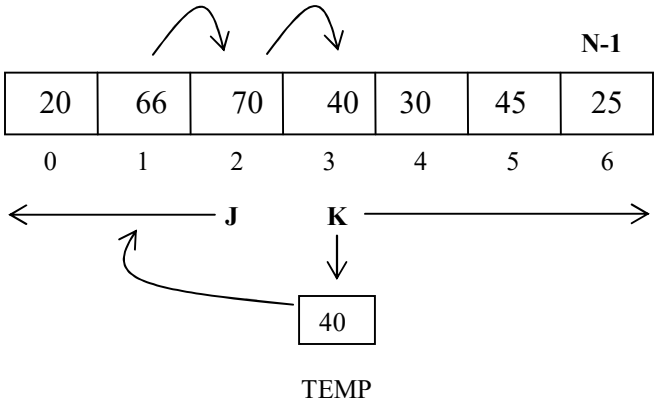
PHASE – I



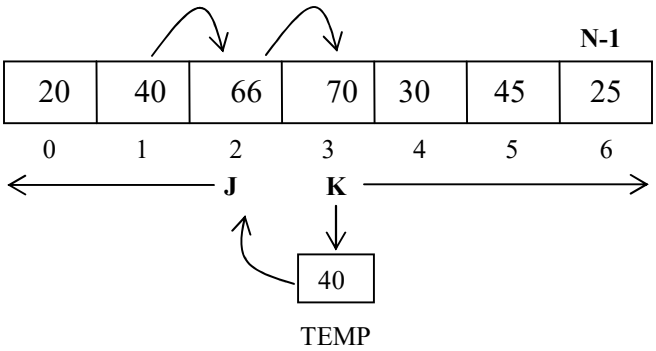
PHASE - II



PHASE - III



PHASE - IV



INSERTION_SORT (A [N])

Step 1: Let J, K, Temp
Step 2: Set K: =1
Step 3: Repeat for K: =1 to N-1 increasing by 1
Set Temp: =A [K]

```
            J: =K-1
        Step3.1: Repeat While (J>=0) AND (Temp < A [J])
                    A [J+1]:=A [J]
                    J: =J-1
                [End of While]
        Step3.2: A [J+1]: = Temp
        [End of For]
Step 4:     Exit
```

22.2 SEARCHING:

Searching is nothing but find the position of an element in the given list of elements. Searching techniques are of two types. They are:

1. Linear Search
2. Binary Search

22.2.1 Linear Search:

When an element is searched in sequence (i.e. from the beginning of the list to till the end of list), it is referred as linear search. Linear search is a time-consuming search especially when the element is present at the end of the list. The search can be either successful (when the element is found at some position) or unsuccessful (if the element is not found in the list) based on the presence or absence of the element in the given list.

Algorithm for Linear Search:
SEARCH (LA, LB, UB, ITEM)
[Assume LA is an array with list of elements, LB and UB represents the lower and upper bounds of the array LA. ITEM is the element to be searched in the list]

```
Step 1:     Let K, FLAG
Step 2: Set K: =LB, FLAG: =0
Step 3:     Repeat While (K<=UB)
        Step 3.1: If LA [K] =ITEM, Then
                Step 3.1.1: Set FLAG: =1
                Step 3.1.2: Display "Item Found at", K
                [End of IF]
```

Step 3.2: Set K**:** =K+1
[End of While]
Step 4: If FLAG=0, Then
Step 4.1: Display "Item is not found"
[End of IF]
Step 5: Exit

22.2.2 Binary Search:

The binary search technique is useful as compared to the linear search but binary search requires the elements in a sorted order. In case of binary search the number of comparisons required to search an element is less as compared to linear search.

Binary search divides the list into two segments by selecting the middle position of the list. As the list is in sorted order, the element to be searched may present either left or right segment of the list based on its value (i.e. if the element is less than middle position value, then the element will be on left segment, otherwise it may present in right segment). This left or right segment is further divided into two segments and this process continues until the element is found or when only one left in the list.

Algorithm for Binary Search:

Let LA is a Linear array with size 20
Search for ITEM = 110

LB UB

10	20	30	40	50	60	70	80	90	100	110	120	130	140	150						
0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20

Begin Mid End

BINARY SEARCH (LA, UB, ITEM)

LA Sorted linear array
LB Lower bound
UB Upper bound
ITEM Element to be searched
LOC Location of ITEM

```
            BEG  Beginning Location of a Segment
            END  End location of a segment
            MID  Middle location of a segment

Step 1:     Let BEG, END, MID, LOC
Step 2:     Set BEG: =LB, END: =UB, MID :=( BEG+END)/2
Step 3:     Repeat While BEG<=END AND LA [MID]! =ITEM
      Step 3.1: If ITEM<LA [MID], Then
                  Set END: =MID-1
      Step 3.2: Else
                  Set BEG: =MID+1
            [End of if]
      Step 3.3: Set MID :=( BEG+END)/2
      [End of While – Step3]

Step 4:     If LA [MID] =ITEM, Then
            Set LOC: =MID
      Else
            Set LOC: =0

      [End of IF]
Step 5:     Exit
```

HASHING

23.1 HASHING

The search concept either it is linear or binary, it is always depends upon the number of elements in the list. As linear search, having complexity O(n) = n and binary search, having time complexity O(n)=long n, which always depends on number of elements 'n'.

Hashing is a searching technique which is independent of the number of elements (say 'n'). So using hashing, searching will be finished within constant period of time.

23.1.1 Hash Table

Whenever a large tabular data is to be stored in memory, there will be unique key values to identify each record. Based on each key value, a hash address will be generated where the corresponding record will be stored.

So hash table contains key values and their hash address/index addresses.

23.1.2 Hash Functions

So Hashing is generating Hash Addresses based on the key values using different mathematical functions called hash functions.

So, if L is the Hash addresses for a key values K
The Hash Function is defined as H: = K → L (can be read as H is mapped from K to L)

There are two main factors for choosing a hash function. They are:

1. The function H should be very easy and quick to compute.
2. The function should give two different indices or hash addresses for two different key values.

Example:

Consider a Hash Table of size 10, where indices (or) hash address are 0,1,2,3,5,6,7,8,9.
Suppose following set of key values exist.
i.e. 10, 19, 35, 43, 62, 59, 31, 49, 77, 33.

The hash function stated here is as follow:

1. Add the two digits of key.
2. Take the digit at unit place of the result as the hash address/index.

Key	Index
10	1
19	0
35	8
43	7
62	8
59	4
31	4
49	3
77	4
33	6

So, the hash table for the given set of key values is as follow:

Index or Hash Address	Key Value
0	19
1	10
2	
3	49
4	51,31,77
5	
6	33
7	43
8	35,62
9	

Here more than one key value for a particular index in the hash table is called collision.

23.1.3 Standard Hash Functions

There are three standard hash functions available which are used to get the hash addresses for the given key values. They are:

1. Division Method
2. Mid Square Method
3. Folding Method

23.1.3.1 Division Method

Choose a number 'M' which is larger than the total number of key values K. Choose 'M' such that it is a prime number or a number without small divisors.

Now to find Hash/index addresses, the Hash function

H (K) = K (mod M) [If index starts with 0]

H (K) = K(mod M) + 1 [If index starts with 1)

e.g. In an organization there are 68 employees having unique employee numbers in 4 digits.

Emp. No	Name	Salary
.	.	
2345	ANIL	12000
.	.	.
.	.	.
3205	MURALI	13000
.	.	.
.	.	.
7145	PRASAD	14000

Here total no. of key values are 68, where some sample key values K=2345, 3205, 7148.

Choose a Prime number 'M', which is larger than 68, say M = 97

Then H(2345) = 2345(mod 97)
= 17

H(3205) =3205(mod 97)
= 4
H(7148) =7148(Mod 97)
= 67

<u>So the Hash Table</u>

Index/Hash Address	**Key Values**
00	
01	
.	
.	
04	3205
. →	
.	
17	2345
. →	
.	
67	7148
. →	
.	
97	

23.1.3.2 Mid - Square Method:

Here the given key value K is squared, and the hash function is defined as: Obtain L by deleting digits from both ends of K^2. Here digits belongs to the same positions must be used for index address. H (K) = L,

Considering the above example of an organization of 68 employees, the hash addresses for the employee numbers can be calculated as follow:

K =	3205	7148	2345
K^2 =	102**72**025	510**93**904	54**99**025

Here the 4th and 5th digits counting from right are chosen for Hash address.

i.e. H(K) = 72 93 99

So, the hash table generated using Mid-square method is as follows:

Index/Hash Address	Key Values
00	
01	
.	
.	
72 →	3205
.	
.	
93 →	7148
.	
.	
99 →	2345
.	
.	

23.1.3.3 Folding Methods

Here key value K is partitioned into no. of parts K_1, K_2, K_3,……Kr, where each part except possibly the last, has the same number of digits as the required address.

Then the parts are added together, by ignoring the last carry. H (K) = $K_1 + K_2 +$ ….+ Kr.

Considering the above example of an organization of 68 employees, the hash addresses for the employee numbers can be calculated as follow:

For K = 3205 H(K) = 32 + 05 = 37
K = 7148 H (K) = 71 + 48 = 119 ≈ 19 (Ignoring the carry)
K = 2345 H (K) = 23 + 45 = 68

Another way of using Folding method is reversing the last partition and adding to the rest part.

For K = 3205 H(K) = 32 + 50 = 82
K = 7148 H (K) = 71 + 84 = 155 ≈ 55 (Ignoring the carry 1)
K = 2345 H (K) = 23 + 54 = 77

23.2 Collision Resolution Techniques:

Consider the following example hash address calculation by using Folding method. Assume the hash function is defined as : Add the two digits of the key and consider unit position digit as index or hash address.

Key Value	Hash Address/ Index
10	1
19	0
35	8
43	7
62	8
59	4
31	4
49	3
77	4
33	6

The hash table for the above is as follow:

Hash Address	Key Value
0	19
1	10
2	
3	49
4	**59, 31,77**
5	
6	33
7	43
8	**35, 62**
9	

Allotment of more than one key value in one location or hash address is called ***collision.***

There are several methods to resolve the collision. Two important methods are:

1. Closed Hashing or Linear Probing or Open Addressing
2. Open Hashing or Chaining

23.2.1 Closed Hashing or Linear Probing:

When each key value is used to generate hash address and to be stored in the address, but when the key value generating a hash address which is already having key value, then it is called collision. In this instance, the key value will be stored in the next free location available in sequential search. i.e. in order of free locations such as I, I + 1, I + 2, ……..h, 1, 2, ……., I – 1

Where 'I' is the hash address location and 'h' is the size of the hash table (i.e. last position of hash table)

The collision that occurred in hash address of previous example can now be solved by following linear probing as follow.

Hash Address	Key Value
0	19
1	10
2	33
3	49
4	59
5	31
6	77
7	43
8	35
9	62

Once key values stored in hash address, the searching for a key value can done in cases as:

1. Until key value is found
2. Until unoccupied locations are encountered
3. Until it reaches to the location where the search has started.

NOTE: Here, the hash table is considered as circular so that when the last location is reached, the search proceeds automatically to first location. That is why this technique is called closed hashing.

Drawbacks of Closed Hashing:

When more number of collisions occurs, the sequential search increases and the searching will become slow.

23.2.1.1 Random Probing:

When the collision occurs at location 'I', the random probing uses a function, which generates a random sequence of locations, rather than an ordered sequence. The random sequence generated by the random number generator contains all positions between '1' and 'h' , where 'h' is the highest location of Hash Table.

For example, if collision exists at I = 2, consider a random number generator:

$$I = (I + M) \% h + 1$$

Where h is highest location of hash table and m is relatively prime to h (means GCD of 'm' and 'h' is 1)

Say M = 5 and h = 11

The sequence of numbers generated by above function is 8,3,9,4,10,5,11,6,1,7,2 . We stop generating when 1st location (i.e. 2) is duplicated.

23.2.1.2 Quadratic Probing:

In quadratic probing, if there is a collision at location ' I' of hash address, then the next location to probed are in the order of $I + 1^2$, $I + 2^2$, $I + 3^2$,…..and so on.

If I = 2, then $I + 1^2 = 3$, $I + 2^2 = 6$, $I + 3^2 = 11$ etc. The key value will be stored in circular nature into the hash address.

NOTE: Quadratic probing is worse than linear probing as there is no guarantee to get free slots.

Drawbacks of Closed Hashing:

The closed hashing has following disadvantages:

1. It is difficult to handle the overflow situation of hash table in satisfactory manner.
2. Key values are haphazardly intermixed due to more probing.

23.2.2 Open Hashing (Address Calculation Sort):

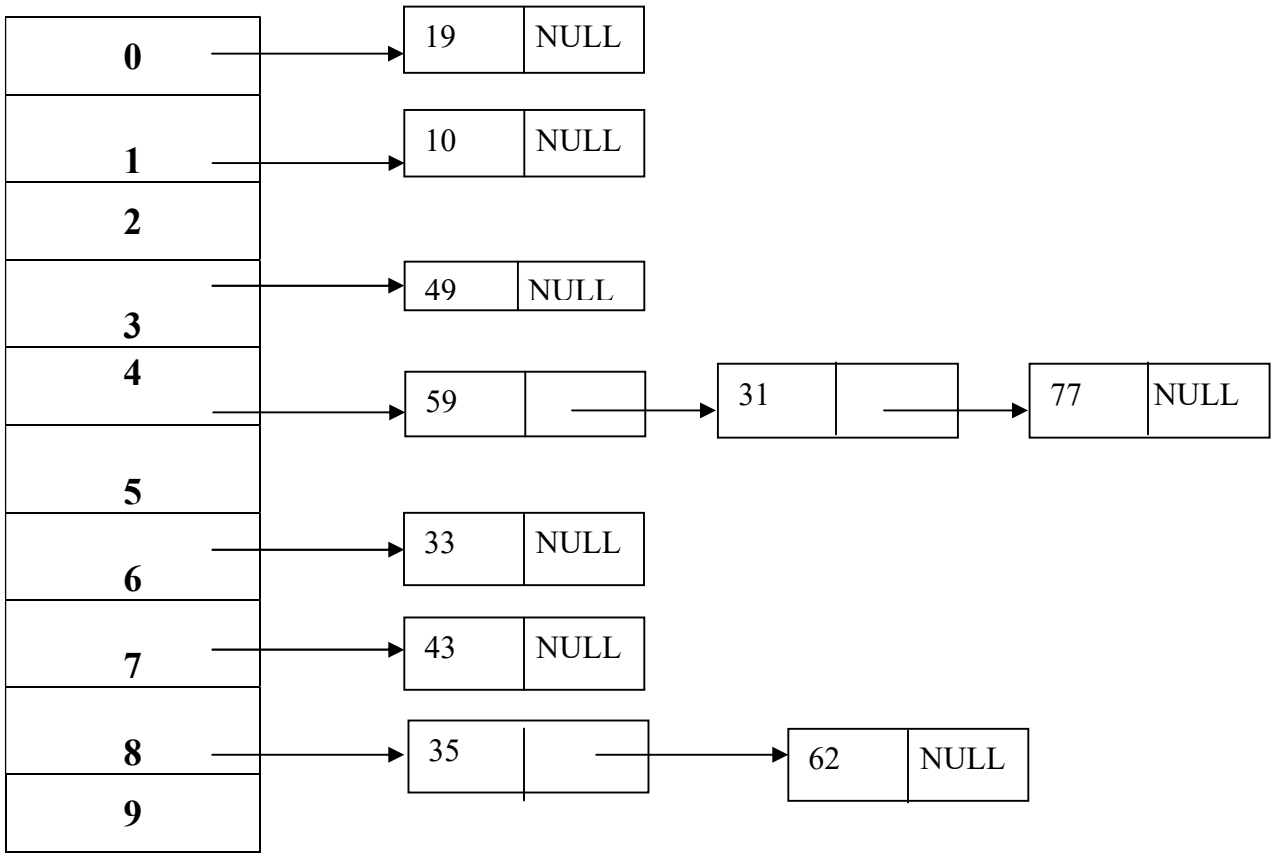
Open hashing is also called separate chaining or address calculation. Here the hash table is treated as array of pointers where each pointer points to a linked list. In this method a hashing function is applied to each key. Depending on the result of hash function the key is placed into one of the linked lists. This way of data representation according to address generated by key is also called as address calculation sort of elements.

Consider the previous example.

Key Value	Hash Address/ Index
10	1
19	0
35	8
43	7
62	8
59	4
31	4
49	3
77	4
33	6

Hash Address	Key Values for linked list
0	19
1	10
2	
3	49
4	59, 31, 77
5	
6	33
7	43
8	35, 62
9	

In open hashing for a given key value, hash address is calculated, then it searches the linked list pointed by the address.



Advantages of Chaining:

1. Overflow situation do not arise in Hash table.
2. Collision resolution can be easily achieved by maintaining linked list in the memory.
3. Insertion and deletion becomes easy.

Overall, we can say that open hashing or chaining is best suitable method to resolve collision as compared to closed hashing.